

Submitted by: Varsha

FOOD DELIVERY

ETA PREDICTION

Predicting Delivery Time using Machine Learning & Deep Learning

ESTIMATED
TIME
ARRIVAL

45,593

Training Records

11,399

Test Records

4

Models Built

0.778

Best R²

Table of Contents

1.	Introduction & Project Overview
2.	Dataset Description
3.	Data Loading & Initial Exploration
4.	Dataset Shape, Columns & Info
5.	Null Values, Duplicates & String NaN Detection
6.	Descriptive Statistics
7.	Data Cleaning & Preprocessing
8.	Missing Value Imputation
9.	Datetime Column Processing
10.	Post-Cleaning Dataset Overview
11.	Exploratory Data Analysis (EDA)
12.	Feature Engineering
13.	Label Encoding & Data Preparation
14.	Train-Test Split & Feature Scaling
15.	Machine Learning Models
16.	Deep Learning — ANN
17.	Model Performance Comparison
18.	Final Predictions on Test Data
19.	Conclusion & Key Takeaways
20.	Thank You & Feedback

1. Introduction & Project Overview

Predicting Food Delivery ETA using ML & Deep Learning

Project Background

In the rapidly growing food delivery industry, accurate ETA prediction is critical for customer satisfaction. This capstone project builds a robust model to estimate delivery time (in minutes) using a complete data science pipeline — from raw data with quality issues to advanced ML and deep learning predictions.

Methodology Overview

Step	Process	Details
1	Data Collection	Two CSV datasets: training (45,593 records) & test (11,399 records)
2	Data Cleaning	Replace string NaN, fix dtypes, impute missing values, parse datetimes
3	EDA	8 visualizations: distributions, box plots, pie charts, scatter plots, heatmap
4	Feature Engineering	Extract order_day, order_month, order_hour from datetime columns
5	Modeling	Train Linear Regression, Random Forest, XGBoost, and ANN models
6	Evaluation	Compare using R2 Score — XGBoost best at R2=0.778

Key Insight	<i>XGBoost achieved R2=0.778 explaining 77.8% of variance. ANN achieved MAE=4.02 min. Festival and traffic density emerged as the strongest ETA predictors.</i>
-------------	---

45,593	11,399	0.778	4.02 min
Training Records	Test Records	Best R2	ANN MAE

2. Dataset Description

Structure and features of the Food Delivery Dataset

About the Dataset

Dataset captures real-world food delivery operations across Indian cities. Includes delivery personnel details, GPS coordinates, order timing, weather, traffic, vehicle types, and actual delivery time. Training set has 20 columns including target Time_taken(min); test set has 19 columns.

Feature	Description	Type
Delivery_person_Age	Age of delivery executive	Numerical
Delivery_person_Ratings	Customer rating (1-6 scale)	Numerical
Restaurant_lat/lon	GPS coordinates of restaurant	Numerical
Delivery_location_lat/lon	GPS coordinates of customer	Numerical
Order_Date	Date order was placed	Datetime
Time_Orderd	Time order was placed	Datetime
Time_Order_picked	Time order was picked up	Datetime
Weatherconditions	Sunny/Stormy/Cloudy/Fog/Windy/Sands torms	Categorical
Road_traffic_density	Low/Medium/High/Jam	Categorical
Vehicle_condition	Vehicle quality (0-3)	Numerical
Type_of_order	Snack/M meal/Drinks/Buffer	Categorical
Type_of_vehicle	Motorcycle/Scooter/Electric/Bicycle	Categorical
multiple_deliveries	Number of simultaneous deliveries	Numerical
Festival	Festival period: Yes/No	Categorical
City	Metropolitan/Urban/Semi-Urban	Categorical
Time_taken(min)	TARGET: Actual delivery time in minutes	Target

Key Insight *Regression problem: target is continuous (10-54 min). Rich mix of categorical, numerical, and datetime features. String "NaN" values disguise missing data — key preprocessing challenge.*

3. Data Loading & Initial Exploration

Loading datasets from Google Drive

3.1 Library Imports

NumPy, Pandas for data manipulation; Seaborn, Matplotlib for visualization; Scikit-learn for ML models and preprocessing; TensorFlow/Keras for ANN; XGBoost for gradient boosting.

```
import numpy as np
import pandas as pd

# Visualization
import seaborn as sns
import matplotlib.pyplot as plt

# Encoding
from sklearn.preprocessing import LabelEncoder

# Train test split
from sklearn.model_selection import train_test_split

# Scaling
from sklearn.preprocessing import StandardScaler

# Neural Network
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Models
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

# Evaluation
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

3.2 Loading Training Data — data_1

Training dataset: 45,593 rows, 20 columns. `head()` shows delivery person details and GPS coordinates.

```
file_id_1 = "1JM2y7CUA80bp2uuVkiUwhIvMctiH2gz"
url = f"https://drive.google.com/uc?export=download&id={file_id_1}"
data_1 = pd.read_csv(url)

data_1.head()
```

	ID	Delivery_person_ID	Delivery_person_Age	Delivery_person_Ratings	Restaurant_latitude	Restaurant_longitude	Delivery_location_latitude	Delivery_location_longitude
0	0x4607	INDORES13DEL02	37	4.9	22.745049	75.892471	22.765049	75.
1	0xb379	BANGRES18DEL02	34	4.5	12.913041	77.683237	13.043041	77.
2	0x5d6d	BANGRES19DEL01	23	4.4	12.914264	77.678400	12.924264	77.
3	0x7a6a	COIMBRES13DEL02	38	4.7	11.003669	76.976494	11.053669	77.
4	0x70a2	CHENRES12DEL01	32	4.6	12.972793	80.249982	13.012793	80.

3.3 Loading Test Data — data_2

Test dataset preview

Test dataset: 11,399 rows, 19 columns — same structure as training except no target column.

```
file_id_2 = "13iMk1QUKq4tUJH9yoMlgCQV4E15v3"
url = f"https://drive.google.com/uc?export=download&id={file_id_2}"
data_2 = pd.read_csv(url)

data_2.head()
```

	ID	Delivery_person_ID	Delivery_person_Age	Delivery_person_Ratings	Restaurant_latitude	Restaurant_longitude	Delivery_location_latitude	Delivery_location_longitude
0	0x2318	COIMBRES13DEL01	NaN	NaN	11.003669	76.976494	11.043669	77.
1	0x3474	BANGRES15DEL01	28	4.6	12.975377	77.696664	13.085377	77.
2	0x9420	JAPRES09DEL03	23	4.5	26.911378	75.789034	27.001378	75.
3	0x72ee	JAPRES07DEL03	21	4.8	26.786536	75.837333	26.856536	75.
4	0xa759	CHENRES19DEL01	31	4.6	12.986047	80.218114	13.096047	80.

Key Insight Training data (~80%) for model building; test data (~20%) for final predictions. Both datasets loaded from Google Drive using file IDs.

4. Dataset Shape, Columns & Info

Structural properties of both datasets

4.1 Training Data Shape — data_1.shape

data_1 has 45,593 entries and 20 columns.

```
: data_1.shape
: (45593, 20)
```

4.2 Training Data Columns

All 20 column names including ID, person details, GPS, datetimes, conditions, vehicle info, target.

```
data_1.columns

Index(['ID', 'Delivery_person_ID', 'Delivery_person_Age',
      'Delivery_person_Ratings', 'Restaurant_latitude',
      'Restaurant_longitude', 'Delivery_location_latitude',
      'Delivery_location_longitude', 'Order_Date', 'Time_Orderd',
      'Time_Order_picked', 'Weatherconditions', 'Road_traffic_density',
      'Vehicle_condition', 'Type_of_order', 'Type_of_vehicle',
      'multiple_deliveries', 'Festival', 'City', 'Time_taken(min)'],
      dtype='object')
```

4.3 Training Data info() — data_1

Data types and non-null counts

Most columns are object dtype. Only 4 truly numeric: float64 coordinates, int64 Vehicle_condition. All 45,593 entries appear non-null — but string "NaN" values hide real missing data.

```
: data_1.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45593 entries, 0 to 45592
Data columns (total 20 columns):
 #   Column                                Non-Null Count  Dtype  
---  -
 0   ID                                    45593 non-null  object  
 1   Delivery_person_ID                   45593 non-null  object  
 2   Delivery_person_Age                  45593 non-null  object  
 3   Delivery_person_Ratings               45593 non-null  object  
 4   Restaurant_latitude                   45593 non-null  float64  
 5   Restaurant_longitude                 45593 non-null  float64  
 6   Delivery_location_latitude            45593 non-null  float64  
 7   Delivery_location_longitude           45593 non-null  float64  
 8   Order_Date                           45593 non-null  object  
 9   Time_Orderd                           45593 non-null  object  
10   Time_Order_picked                     45593 non-null  object  
11   Weatherconditions                     45593 non-null  object  
12   Road_traffic_density                  45593 non-null  object  
13   Vehicle_condition                     45593 non-null  int64  
14   Type_of_order                         45593 non-null  object  
15   Type_of_vehicle                       45593 non-null  object  
16   multiple_deliveries                  45593 non-null  object  
17   Festival                             45593 non-null  object  
18   City                                 45593 non-null  object  
19   Time_taken(min)                       45593 non-null  object  
dtypes: float64(4), int64(1), object(15)
memory usage: 7.0+ MB
```

4.4 Test Data Shape, Columns & info() — data_2

Structure of test dataset

4.4.1 Test Data Shape

```
In [ ]: data_2.shape  
Out[ ]: (11399, 19)
```

4.4.2 Test Data Columns

```
data_2.columns  
Index(['ID', 'Delivery_person_ID', 'Delivery_person_Age',  
       'Delivery_person_Ratings', 'Restaurant_latitude',  
       'Restaurant_longitude', 'Delivery_location_latitude',  
       'Delivery_location_longitude', 'Order_Date', 'Time_Orderd',  
       'Time_Order_picked', 'Weatherconditions', 'Road_traffic_density',  
       'Vehicle_condition', 'Type_of_order', 'Type_of_vehicle',  
       'multiple_deliveries', 'Festival', 'City'],  
      dtype='object')
```

4.4.3 Test Data info()

11,399 entries, 19 columns — same structure as training except no target column Time_taken(min).


```
data_2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 11399 entries, 0 to 11398
```

```
Data columns (total 19 columns):
```

#	Column	Non-Null Count	Dtype
0	ID	11399 non-null	object
1	Delivery_person_ID	11399 non-null	object
2	Delivery_person_Age	11399 non-null	object
3	Delivery_person_Ratings	11399 non-null	object
4	Restaurant_latitude	11399 non-null	float64
5	Restaurant_longitude	11399 non-null	float64
6	Delivery_location_latitude	11399 non-null	float64
7	Delivery_location_longitude	11399 non-null	float64
8	Order_Date	11399 non-null	object
9	Time_Orderd	11399 non-null	object
10	Time_Order_picked	11399 non-null	object
11	Weatherconditions	11399 non-null	object
12	Road_traffic_density	11399 non-null	object
13	Vehicle_condition	11399 non-null	int64
14	Type_of_order	11399 non-null	object
15	Type_of_vehicle	11399 non-null	object
16	multiple_deliveries	11399 non-null	object
17	Festival	11399 non-null	object
18	City	11399 non-null	object

```
dtypes: float64(4), int64(1), object(14)
```

```
memory usage: 1.7+ MB
```

Key Insight

Dominance of "object" dtype is root cause of hidden missing values. Standard `isnull()` returns zero — literal "NaN" strings exist throughout. Must resolve before imputation.

5. Null Values, Duplicates & String NaN Detection

Uncovering hidden missing data

5.1 Initial Null Check — data_1

`isnull().sum()` on `data_1` shows zero nulls — misleading because missing values are stored as string "NaN".

```
data_1.isnull().sum()

ID                                0
Delivery_person_ID               0
Delivery_person_Age              0
Delivery_person_Ratings          0
Restaurant_latitude              0
Restaurant_longitude             0
Delivery_location_latitude       0
Delivery_location_longitude     0
Order_Date                      0
Time_Orderd                     0
Time_Order_picked               0
Weatherconditions               0
Road_traffic_density            0
Vehicle_condition               0
Type_of_order                   0
Type_of_vehicle                 0
multiple_deliveries             0
Festival                        0
City                           0
Time_taken(min)                 0
dtype: int64
```

5.2 Duplicate Check — data_1

`data_1.duplicated().sum()>0` returns `array([False])` — no duplicates found.

```
data_1.duplicated().sum()>[0]

array([False])
```

5.3 Null & Duplicate Check — data_2

Test data quality verification

5.3.1 Null Check — data_2

`isnull().sum()` on `data_2` also shows zero nulls — same misleading result due to string NaN.

```
data_2.isnull().sum()

ID                                0
Delivery_person_ID               0
Delivery_person_Age              0
Delivery_person_Ratings          0
Restaurant_latitude              0
Restaurant_longitude             0
Delivery_location_latitude       0
Delivery_location_longitude     0
Order_Date                      0
Time_Orderd                     0
Time_Order_picked               0
Weatherconditions               0
Road_traffic_density            0
Vehicle_condition               0
Type_of_order                   0
Type_of_vehicle                 0
multiple_deliveries             0
Festival                        0
City                           0
dtype: int64
```

5.3.2 Duplicate Check — data_2

`data_2.duplicated().sum()>0` returns `array([False])` — no duplicates in test data.

```
data_2.duplicated().sum()>0

array([False])
```

5.4 String "NaN" Detection — Training Data

Finding hidden missing values in data_1

Checking `(data_1=="NaN").sum()` reveals true missing data: Age:1,854 | Ratings:1,908 | Time_Orderd:1,731 | Traffic:601 | multiple_deliveries:993 | Festival:228 | City:1,200 | Time_taken:1,200.

```
(data_1 == 'NaN').sum()
```

```
ID          0
Delivery_person_ID  0
Delivery_person_Age    1854
Delivery_person_Ratings  1908
Restaurant_latitude    0
Restaurant_longitude    0
Delivery_location_latitude  0
Delivery_location_longitude  0
Order_Date          0
Time_Orderd        1731
Time_Order_picked    0
Weatherconditions    0
Road_traffic_density    601
Vehicle_condition    0
Type_of_order        0
Type_of_vehicle      0
multiple_deliveries    993
Festival            228
City               1200
Time_taken(min)      0
dtype: int64
```

Key Insight

Standard null detection completely fails — missing values encoded as literal string "NaN". All must be replaced with `numpy.nan` before imputation can proceed.

5.5 String "NaN" Detection — Test Data

Hidden missing values in data_2

Checking `(data_2=="NaN").sum()`: Age:491 | Ratings:507 | Time_Orderd:444 | Traffic:154 | multiple_deliveries:238 | Festival:65 | City:324.

```
(data_2 == 'NaN').sum()
```

```
ID      0
Delivery_person_ID  0
Delivery_person_Age  491
Delivery_person_Ratings  507
Restaurant_latitude  0
Restaurant_longitude  0
Delivery_location_latitude  0
Delivery_location_longitude  0
Order_Date  0
Time_Orderd  444
Time_Order_picked  0
Weatherconditions  0
Road_traffic_density  154
Vehicle_condition  0
Type_of_order  0
Type_of_vehicle  0
multiple_deliveries  238
Festival  65
City  324
dtype: int64
```

Key Insight

Both datasets have significant hidden missing values as literal strings. Same cleaning strategy — replace "NaN" string with `numpy.nan` — applied to both.

6. Descriptive Statistics

Statistical summary of numerical features

6.1 Training Data — `data_1.describe()`

Count, mean, std, min, quartiles, max. Coordinates centered around India (lat~17, lon~70-78).
Vehicle_condition: 0-3.

<code>data_1.describe()</code>					
	Restaurant_latitude	Restaurant_longitude	Delivery_location_latitude	Delivery_location_longitude	Vehicle_condition
count	45593.000000	45593.000000	45593.000000	45593.000000	45593.000000
mean	17.017729	70.231332	17.465186	70.845702	1.023359
std	8.185109	22.883647	7.335122	21.118812	0.839065
min	-30.905562	-88.366217	0.010000	0.010000	0.000000
25%	12.933284	73.170000	12.988453	73.280000	0.000000
50%	18.546947	75.898497	18.633934	76.002574	1.000000
75%	22.728163	78.044095	22.785049	78.107044	2.000000
max	30.914057	88.433452	31.054057	88.563452	3.000000

6.2 Test Data — `data_2.describe()`

Nearly identical distributions to training data — same means and ranges. No distribution shift. Model should generalize well.

<code>data_2.describe()</code>					
	Restaurant_latitude	Restaurant_longitude	Delivery_location_latitude	Delivery_location_longitude	Vehicle_condition
count	11399.000000	11399.000000	11399.000000	11399.000000	11399.000000
mean	17.099934	70.399259	17.569497	71.102187	1.031406
std	8.193510	22.773144	7.287440	20.693782	0.839599
min	-30.902872	-88.400467	0.010000	0.010000	0.000000
25%	12.933284	73.170937	12.992532	73.771081	0.000000
50%	18.551440	75.897429	18.643481	75.996959	1.000000
75%	22.732225	78.045732	22.791226	78.109004	2.000000
max	30.914057	88.433452	31.054057	88.563452	3.000000

Key Insight

Matching statistical distributions confirms both sets are from same population. No outlier issues in coordinates. Vehicle_condition consistently 0-3 in both sets.

7. Data Cleaning & Preprocessing

Fixing data types and replacing string NaN values

7.1 Replace String "NaN" with numpy.nan

Three columns (Age, Ratings, multiple_deliveries) had string "NaN" replaced with numpy.nan. Same applied to data_2.

```
cols = ['Delivery_person_Age',  
        'Delivery_person_Ratings',  
        'multiple_deliveries']  
data_1[cols] = data_1[cols].replace('NaN ', np.nan)
```

7.2 Drop ID Column

ID column has no predictive value — dropped from training data to prevent identity leakage.

```
data_1.drop('ID', axis=1, inplace=True)
```

7.3 Column 1 — Delivery_person_Age: object to float64

Age stored as object with numeric strings. After NaN replacement, converted to float64. Range: 15-50 years.

column 1

```
data_1["Delivery_person_Age"].dtype
```

```
dtype('O')
```

```
data_1['Delivery_person_Age'].unique()
```

```
array(['37', '34', '23', '38', '32', '22', '33', '35', '36', '21', '24',  
      '29', '25', '31', '27', '26', '20', nan, '28', '39', '30', '15',  
      '50'], dtype=object)
```

```
data_1['Delivery_person_Age'] = data_1['Delivery_person_Age'].astype(float)
```

```
data_1['Delivery_person_Age'].dtype
```

```
dtype('float64')
```

```
data_1['Delivery_person_Age'].unique()
```

```
array([37., 34., 23., 38., 32., 22., 33., 35., 36., 21., 24., 29., 25.,  
      31., 27., 26., 20., nan, 28., 39., 30., 15., 50.])
```

7. Data Cleaning (Continued)

Columns 2, 3, and 4 type conversions

7.4 Column 2 — `Delivery_person_Ratings`: object to float64

Ratings stored as object. Converted to float64 after NaN replacement. Range: 1.0 to 6.0.

```
Column 2

: data_1["Delivery_person_Ratings"].dtype
: dtype('O')

: data_1["Delivery_person_Ratings"].unique()
: array(['4.9', '4.5', '4.4', '4.7', '4.6', '4.8', '4.2', '4.3', '4', '4.1',
        '5', '3.5', nan, '3.8', '3.9', '3.7', '2.6', '2.5', '3.6', '3.1',
        '2.7', '1', '3.2', '3.3', '6', '3.4', '2.8', '2.9', '3'],
        dtype=object)

: data_1["Delivery_person_Ratings"] = data_1["Delivery_person_Ratings"].astype(float)

: data_1["Delivery_person_Ratings"].dtype
: dtype('float64')

: data_1["Delivery_person_Ratings"].unique()
: array([4.9, 4.5, 4.4, 4.7, 4.6, 4.8, 4.2, 4.3, 4. , 4.1, 5. , 3.5, nan,
        3.8, 3.9, 3.7, 2.6, 2.5, 3.6, 3.1, 2.7, 1. , 3.2, 3.3, 6. , 3.4,
        2.8, 2.9, 3. ])
```

7.5 Column 3 — `multiple_deliveries`: object to float64

Discrete column (0,1,2,3) stored as object. Converted to float64 for mode imputation.

Column 3

```
In [ ]: data_1["multiple_deliveries"].dtype
Out[ ]: dtype('O')

In [ ]: data_1['multiple_deliveries'].unique()
Out[ ]: array(['0', '1', '3', nan, '2'], dtype=object)

In [ ]: data_1['multiple_deliveries'] = data_1['multiple_deliveries'].astype(float)

In [ ]: data_1["multiple_deliveries"].dtype
Out[ ]: dtype('float64')

In [ ]: data_1['multiple_deliveries'].unique()
Out[ ]: array([ 0.,  1.,  3., nan,  2.])
```

7.6 Column 4 — Time_taken(min): Target Variable Cleaning

Values like "(min) 24" — "(min) " prefix stripped using `str.replace()`, then converted to `int64`. Range: 10-54 min.

Column 4

```
In [ ]: data_1['Time_taken(min)'].dtype
Out[ ]: dtype('O')

In [ ]: data_1['Time_taken(min)'].unique()
Out[ ]: array(['(min) 24', '(min) 33', '(min) 26', '(min) 21', '(min) 30',
              '(min) 40', '(min) 32', '(min) 34', '(min) 46', '(min) 23',
              '(min) 20', '(min) 41', '(min) 15', '(min) 36', '(min) 39',
              '(min) 18', '(min) 38', '(min) 47', '(min) 12', '(min) 22',
              '(min) 25', '(min) 35', '(min) 10', '(min) 19', '(min) 11',
              '(min) 28', '(min) 52', '(min) 16', '(min) 27', '(min) 49',
              '(min) 17', '(min) 14', '(min) 37', '(min) 44', '(min) 42',
              '(min) 31', '(min) 13', '(min) 29', '(min) 50', '(min) 43',
              '(min) 48', '(min) 54', '(min) 53', '(min) 45', '(min) 51'],
              dtype=object)

In [ ]: data_1['Time_taken(min)'] = data_1['Time_taken(min)'].str.replace('(min) ', '', regex=False)

In [ ]: data_1['Time_taken(min)'] = data_1['Time_taken(min)'].astype(int)

In [ ]: data_1['Time_taken(min)'].dtype
Out[ ]: dtype('int64')

In [ ]: data_1['Time_taken(min)'].unique()
Out[ ]: array([24, 33, 26, 21, 30, 40, 32, 34, 46, 23, 20, 41, 15, 36, 39, 18, 38,
              47, 12, 22, 25, 35, 10, 19, 11, 28, 52, 16, 27, 49, 17, 14, 37, 44,
              42, 31, 13, 29, 50, 43, 48, 54, 53, 45, 51])
```

Key Insight

The "(min) 24" format required string manipulation before type conversion. This is a real-world data entry issue. Removing ID prevents row identity leaking into predictions.

8. Missing Value Imputation

Filling null values using median and mode

Strategy: Median for continuous columns (Age, Ratings) — robust to outliers. Mode for discrete column (multiple_deliveries). Applied identically to both data_1 and data_2.

8.1 Training Data Imputation — data_1

```
data_1['Delivery_person_Age'].fillna(data_1["Delivery_person_Age"].median(),inplace = True)
data_1['Delivery_person_Ratings'].fillna(data_1["Delivery_person_Ratings"].median(),inplace = True)
data_1['multiple_deliveries'].fillna(data_1["multiple_deliveries"].mode()[0],inplace = True)
```

8.2 Null Check After Imputation — data_1

After imputation, all columns confirm zero null values in training data.

```
data_1.isnull().sum()

Delivery_person_ID      0
Delivery_person_Age      0
Delivery_person_Ratings  0
Restaurant_latitude      0
Restaurant_longitude     0
Delivery_location_latitude 0
Delivery_location_longitude 0
Order_Date              0
Time_Orderd             0
Time_Order_picked       0
Weatherconditions       0
Road_traffic_density     0
Vehicle_condition       0
Type_of_order           0
Type_of_vehicle         0
multiple_deliveries     0
Festival                0
City                    0
Time_taken(min)         0
dtype: int64
```

8. Missing Value Imputation (Continued)

Test data imputation and verification

8.3 Test Data Imputation — data_2

```
data_2['Delivery_person_Age'].fillna(data_2["Delivery_person_Age"].median(),inplace = True)
data_2['Delivery_person_Ratings'].fillna(data_2["Delivery_person_Ratings"].median(),inplace = True)
data_2['multiple_deliveries'].fillna(data_2["multiple_deliveries"].mode()[0],inplace = True)
```

8.4 Null Check After Imputation — data_2

After imputation, all columns show zero null values in test data.

```
data_2.isnull().sum()

Delivery_person_ID      0
Delivery_person_Age      0
Delivery_person_Ratings  0
Restaurant_latitude      0
Restaurant_longitude     0
Delivery_location_latitude 0
Delivery_location_longitude 0
Order_Date              0
Time_Orderd             0
Time_Order_picked       0
Weatherconditions       0
Road_traffic_density     0
Vehicle_condition       0
Type_of_order           0
Type_of_vehicle         0
multiple_deliveries     0
Festival                0
City                    0
dtype: int64
```

Key Insight

*Median imputation chosen over mean — skewed distributions distort the mean.
Mode appropriate for multiple_deliveries as it is a discrete count variable.*

8.5 Post-Cleaning Dataset Info

Verifying final clean structure and data types

Training Data info() After Cleaning — data_1

45,593 entries | 19 columns (ID dropped) | Age/Ratings/multiple_deliveries now float64 | Time_taken int64 | Zero nulls.

```
data_1.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45593 entries, 0 to 45592
Data columns (total 19 columns):
 #   Column                                Non-Null Count  Dtype  
---  --
 0   Delivery_person_ID                   45593 non-null  object 
 1   Delivery_person_Age                  45593 non-null  float64
 2   Delivery_person_Ratings              45593 non-null  float64
 3   Restaurant_latitude                  45593 non-null  float64
 4   Restaurant_longitude                 45593 non-null  float64
 5   Delivery_location_latitude           45593 non-null  float64
 6   Delivery_location_longitude          45593 non-null  float64
 7   Order_Date                           45593 non-null  object 
 8   Time_Orderd                          45593 non-null  object 
 9   Time_Order_picked                    45593 non-null  object 
10   Weatherconditions                    45593 non-null  object 
11   Road_traffic_density                 45593 non-null  object 
12   Vehicle_condition                    45593 non-null  int64  
13   Type_of_order                        45593 non-null  object 
14   Type_of_vehicle                      45593 non-null  object 
15   multiple_deliveries                  45593 non-null  float64
16   Festival                             45593 non-null  object 
17   City                                 45593 non-null  object 
18   Time_taken(min)                      45593 non-null  int64  
dtypes: float64(7), int64(2), object(10)
memory usage: 6.6+ MB
```

Test Data info() After Cleaning — data_2

11,399 entries | 18 columns | Same dtype corrections applied | All null values imputed.

```
data_2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 11399 entries, 0 to 11398
```

```
Data columns (total 18 columns):
```

#	Column	Non-Null Count	Dtype
0	Delivery_person_ID	11399 non-null	object
1	Delivery_person_Age	11399 non-null	float64
2	Delivery_person_Ratings	11399 non-null	float64
3	Restaurant_latitude	11399 non-null	float64
4	Restaurant_longitude	11399 non-null	float64
5	Delivery_location_latitude	11399 non-null	float64
6	Delivery_location_longitude	11399 non-null	float64
7	Order_Date	11399 non-null	object
8	Time_Orderd	11399 non-null	object
9	Time_Order_picked	11399 non-null	object
10	Weatherconditions	11399 non-null	object
11	Road_traffic_density	11399 non-null	object
12	Vehicle_condition	11399 non-null	int64
13	Type_of_order	11399 non-null	object
14	Type_of_vehicle	11399 non-null	object
15	multiple_deliveries	11399 non-null	float64
16	Festival	11399 non-null	object
17	City	11399 non-null	object

```
dtypes: float64(7), int64(1), object(10)
```

```
memory usage: 1.6+ MB
```

Key Insight

Complete cleaning pipeline transformed raw messy data with hidden NaN strings into model-ready format with correct data types and zero missing values.

9. Datetime Column Processing

Converting date/time strings to datetime objects

9.1 Order_Date Conversion

Order_Date stored as "DD-MM-YYYY" string. Converted to datetime64[ns] using `pd.to_datetime()`. Enables extraction of day and month components for feature engineering.

```
Date Column

In [ ]: data_1['Order_Date'].head()

Out[ ]:
0    19-03-2022
1    25-03-2022
2    19-03-2022
3    05-04-2022
4    26-03-2022
Name: Order_Date, dtype: object

In [ ]: data_1['Order_Date'] = pd.to_datetime(data_1['Order_Date'])

C:\Users\HP\AppData\Local\Temp\ipykernel_16508\3835211906.py:1: UserWarning:
data_1['Order_Date'] = pd.to_datetime(data_1['Order_Date'])

In [ ]: data_1['Order_Date'].head()

Out[ ]:
0    2022-03-19
1    2022-03-25
2    2022-03-19
3    2022-04-05
4    2022-03-26
Name: Order_Date, dtype: datetime64[ns]
```

9.2 Time_Orderd & Time_Order_picked Conversion

Both time columns converted from string to datetime. String NaN replaced first. Missing Time_Orderd values filled using mode. Same process applied to data_2.

```
Time Column

In [ ]: data_1['Time_Orderd'].head()

Out[ ]:
0    11:30:00
1    19:45:00
2    08:30:00
3    18:00:00
4    13:30:00
Name: Time_Orderd, dtype: object

In [ ]: data_1['Time_Order_picked'].head()

Out[ ]:
0    11:45:00
1    19:50:00
2    08:45:00
3    18:10:00
4    13:45:00
Name: Time_Order_picked, dtype: object

In [ ]: data_1['Time_Orderd'] = data_1['Time_Orderd'].replace('NaN ', np.nan)

In [ ]: data_1['Time_Orderd'] = pd.to_datetime(data_1['Time_Orderd'])
In [ ]: data_1['Time_Order_picked'] = pd.to_datetime(data_1['Time_Order_picked'])
```

9.3 After Datetime Conversion & Null Check

Verifying successful conversion

Time_Orderd After Conversion — data_1

Both columns now in datetime64[ns] format showing proper 2026-05-23 timestamps.

```
: data_1['Time_Orderd'].head()

: 0    2026-05-23 11:30:00
  1    2026-05-23 19:45:00
  2    2026-05-23 08:30:00
  3    2026-05-23 18:00:00
  4    2026-05-23 13:30:00
  Name: Time_Orderd, dtype: datetime64[ns]

: data_1['Time_Order_picked'].head()

: 0    2026-05-23 11:45:00
  1    2026-05-23 19:50:00
  2    2026-05-23 08:45:00
  3    2026-05-23 18:10:00
  4    2026-05-23 13:45:00
  Name: Time_Order_picked, dtype: datetime64[ns]
```

9.4 Final Null Check on Datetime Columns — Both Datasets

After all datetime conversions and mode imputation, Order_Date/Time_Orderd/Time_Order_picked confirm zero nulls in both data_1 and data_2.


```
data_1[['Order_Date', 'Time_Orderd', 'Time_Order_picked']].isnull().sum()

Order_Date      0
Time_Orderd    1731
Time_Order_picked  0
dtype: int64

data_2[['Order_Date', 'Time_Orderd', 'Time_Order_picked']].isnull().sum()

Order_Date      0
Time_Orderd     444
Time_Order_picked  0
dtype: int64

data_1['Time_Orderd'].fillna(data_1['Time_Orderd'].mode()[0], inplace=True)
data_2['Time_Orderd'].fillna(data_2['Time_Orderd'].mode()[0], inplace=True)

C:\Users\HP\AppData\Local\Temp\ipykernel_16508\594225965.py:1: FutureWarning:

data_1[['Order_Date', 'Time_Orderd', 'Time_Order_picked']].isnull().sum()

Order_Date      0
Time_Orderd      0
Time_Order_picked  0
dtype: int64

data_2[['Order_Date', 'Time_Orderd', 'Time_Order_picked']].isnull().sum()

Order_Date      0
Time_Orderd      0
Time_Order_picked  0
dtype: int64
```

Key Insight *Datetime conversion unlocks temporal feature engineering. order_hour (peak hours 12-14, 19-22) is the most valuable new feature — without datetime parsing this predictor would be lost.*

11. Exploratory Data Analysis (EDA)

Understanding patterns through rich visualizations

11.1 Distribution of Delivery Time

Histogram of Time_taken(min) shows range 10-54 minutes. Right-skewed distribution with peak frequency around 25-28 minutes. Red dashed line marks the mean at approximately 27-28 minutes.



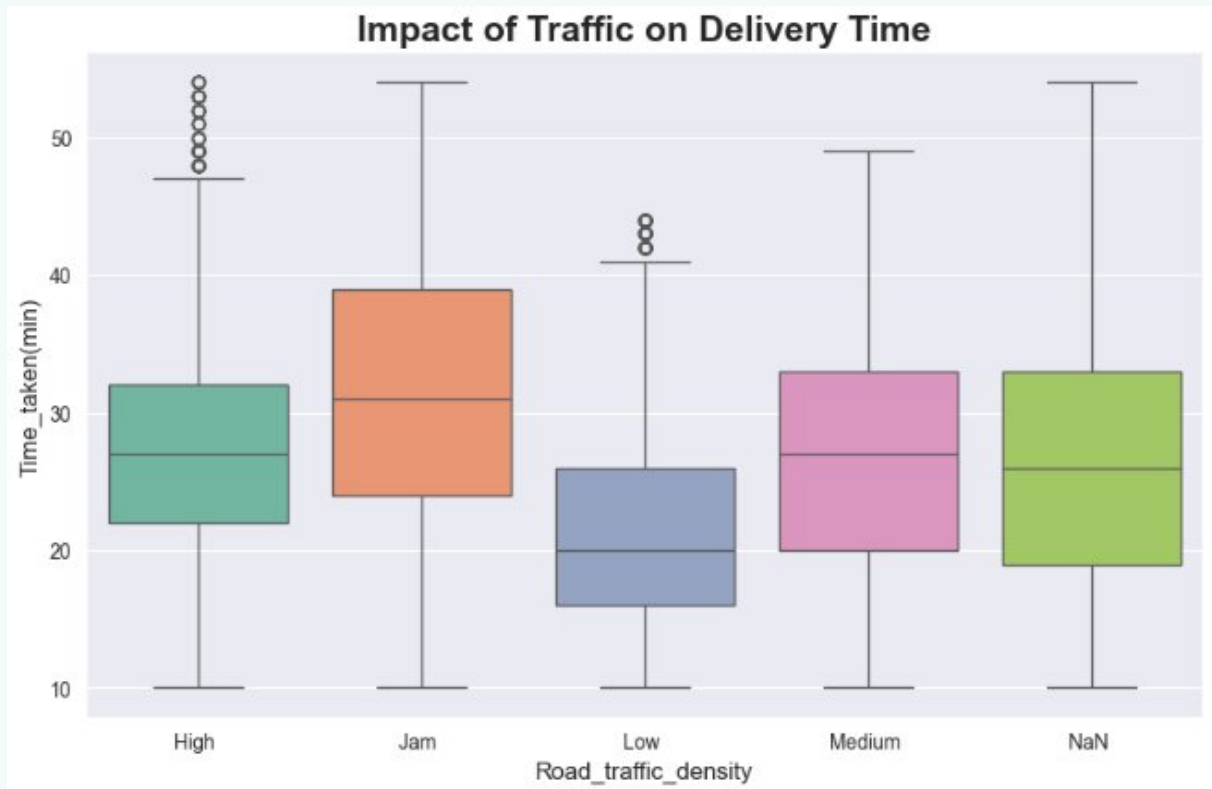
Key Insight

Most deliveries complete in 20-30 minutes. Right skew indicates some deliveries take much longer due to traffic jams, festivals, or adverse weather. Bimodal pattern hints at two distinct speed groups — likely correlated with city type or traffic density.

11.2 Impact of Traffic Density on Delivery Time

Road conditions vs delivery speed

"Jam" traffic causes highest median delivery time (~30 min) with widest spread. "Low" traffic achieves fastest deliveries (median ~20 min). "High" and "Medium" both around 27 minutes. NaN category shows distribution similar to Medium traffic.



Key Insight

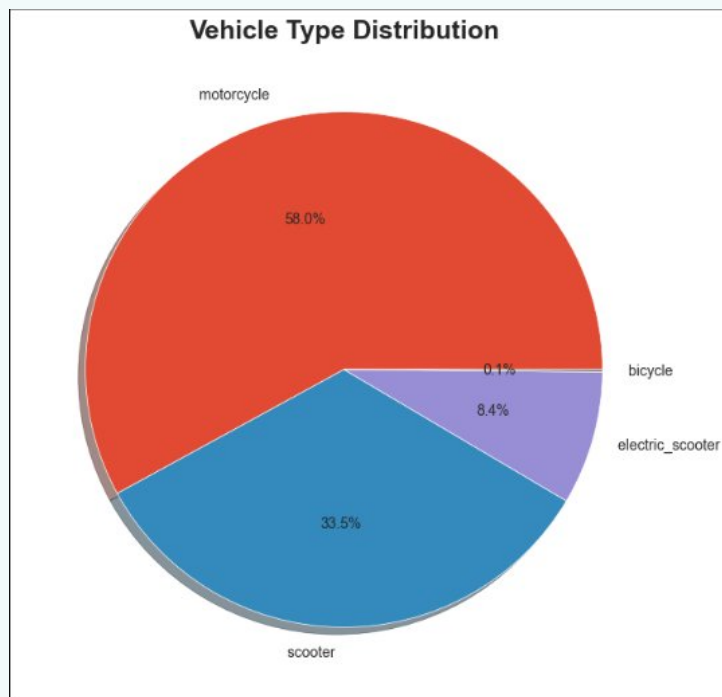
Traffic jam conditions increase delivery times by 40-50% vs low-traffic. Road_traffic_density is one of the strongest predictors in the model — this feature alone accounts for a major portion of delivery time variance.

11.3 Vehicle Type Distribution & Ratings Analysis

Fleet composition and ratings impact

11.3.1 Vehicle Type Pie Chart

Motorcycles: 58.0% | Scooters: 33.5% | Electric scooters: 8.4% | Bicycles: 0.1%. Reflects the delivery landscape in Indian urban and metropolitan areas.

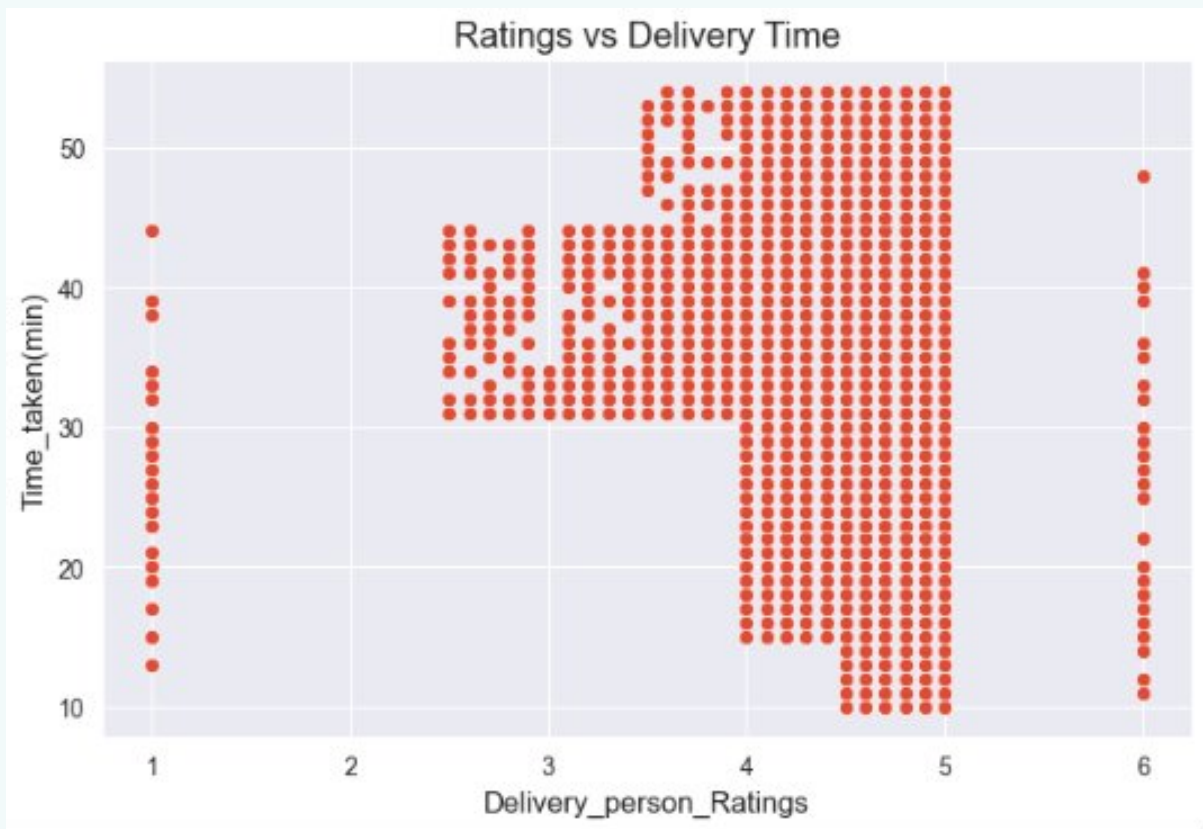


Key Insight

Motorcycle dominance (58%) means Type_of_vehicle has limited discriminatory power. Bicycles (0.1%) — though rare — likely have longest delivery times due to speed constraints.

11.3.2 Delivery Person Ratings vs Delivery Time — Scatter Plot

No strong linear correlation between ratings and delivery speed. Most deliveries cluster at 4.0-5.0 ratings covering full time range (10-55 min). Ratings measure quality, not speed.



Key Insight

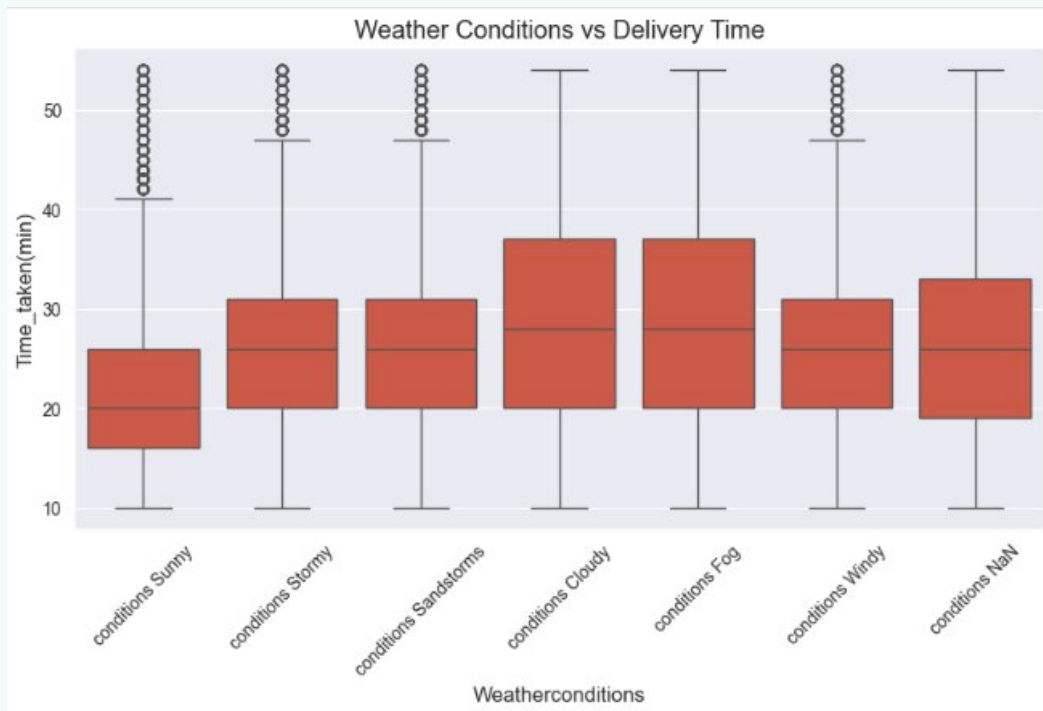
Higher-rated delivery personnel are not necessarily faster — they are simply more courteous. Near-zero correlation confirms ratings measure customer satisfaction, not delivery efficiency.

11.4 Weather Conditions & Festival Impact

Environmental and seasonal factors

11.4.1 Weather Conditions vs Delivery Time

Cloudy and Fog: highest median times (~27-28 min). Sunny: fastest (median ~20 min). Sandstorm and Stormy: high variance across deliveries.



Key Insight

Fog and Cloudy conditions force delivery personnel to slow down for safety. Stormy conditions show high variance — some deliveries proceed despite harsh weather while others face severe delays.

11.4.2 Festival Impact on Delivery Time

Festival periods show dramatic delays — median nearly doubles from ~25 min (normal) to ~45 minutes. Festival is the single most impactful categorical feature.



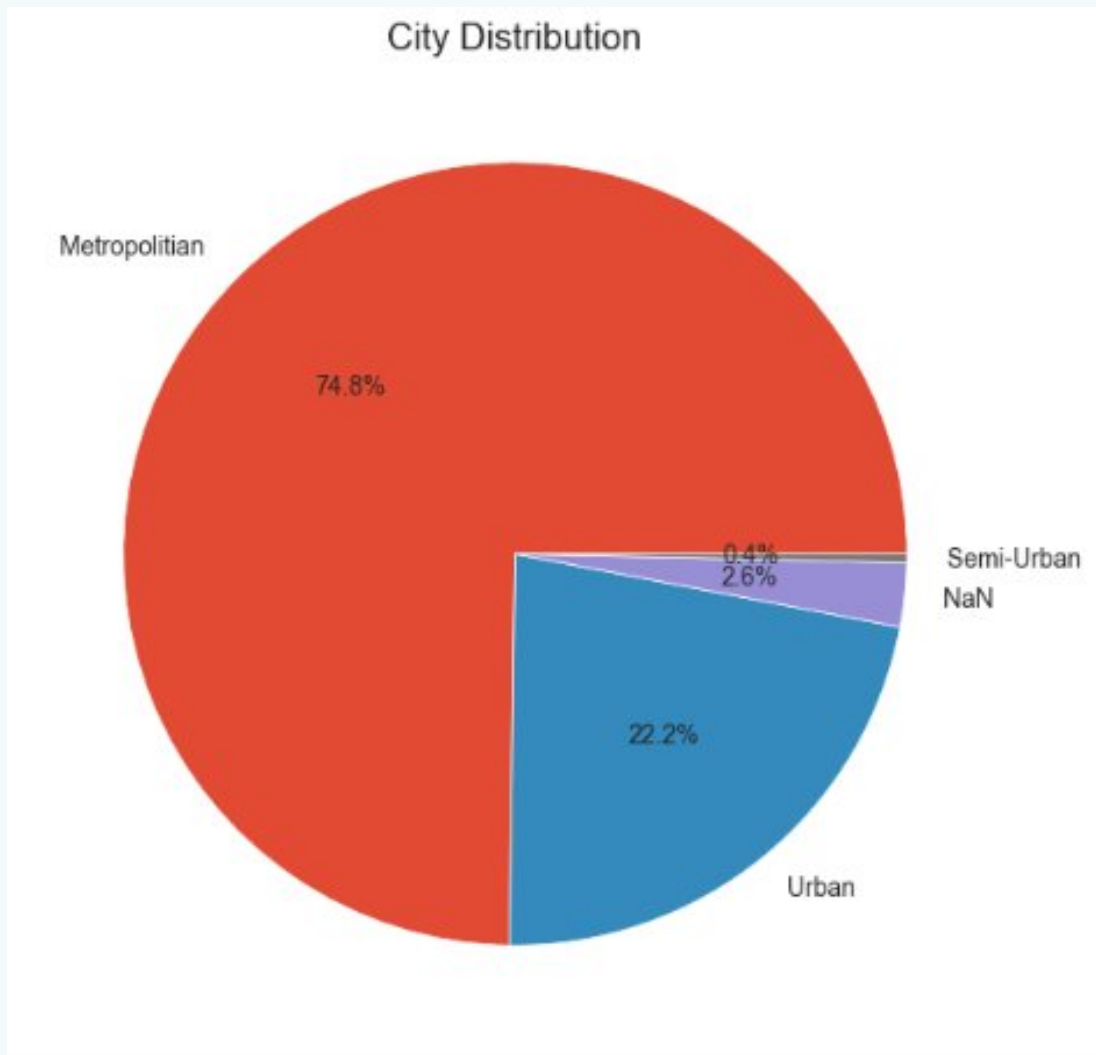
Key Insight *80% increase in median delivery time during festivals reflects increased order volumes, road congestion, and reduced delivery person availability. Festival feature is critical for accuracy.*

11.5 City Distribution

Geographic delivery patterns

City Distribution Pie Chart

Metropolitan: 74.8% | Urban: 22.2% | Semi-Urban: 0.4%. Model is primarily optimized for metropolitan delivery scenarios.



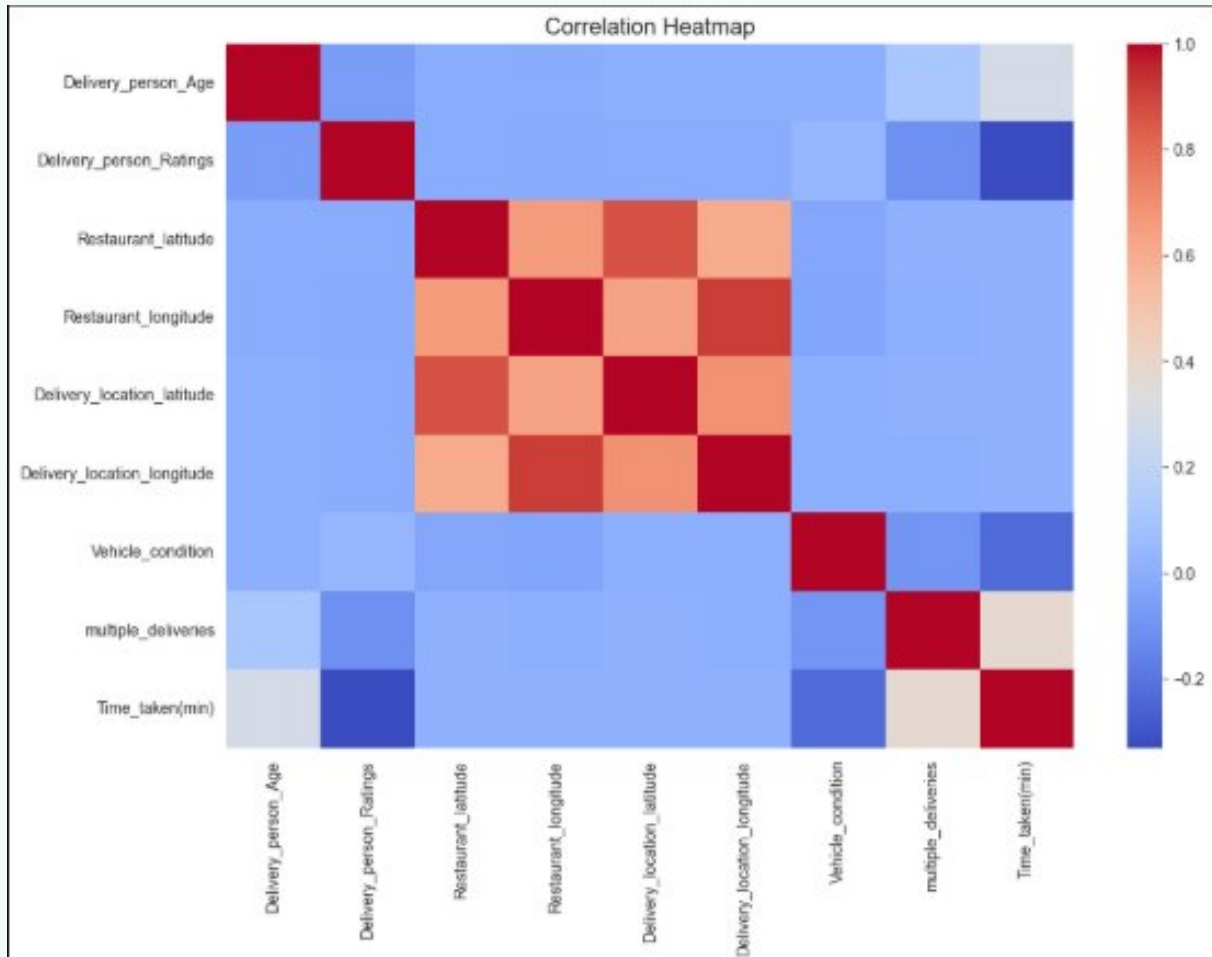
Key Insight

Semi-Urban deliveries (0.4%) are rare — model may underperform for this category. City type captures infrastructure quality, road density, and overall delivery conditions.

11.6 Correlation Heatmap

Linear relationships between all numerical features

Pearson correlation matrix. Strong positive (dark red ~1.0) between restaurant and delivery coordinates — expected for local deliveries. Negative (blue) for multiple_deliveries vs Time_taken.



Key Insight (1) Restaurant/delivery coordinates strongly correlated — always local deliveries. (2) multiple_deliveries has negative correlation with delivery time. (3) Age and Ratings show near-zero correlation — they measure quality, not speed.

Feature Pair	Correlation	Interpretation
Restaurant lat vs Delivery lat	~0.90+	Strong — always local deliveries
multiple_deliveries vs Time	Negative	Batch deliveries alter time pattern
Age vs Time_taken	Near zero	Age does not predict delivery speed
Ratings vs Time_taken	Near zero	Ratings measure quality not speed

12. Feature Engineering

Extracting temporal features from datetime columns

12.1 Identifying Column Categories

Categorical (object dtype), Numerical (int64/float64), Datetime (datetime64). Target Time_taken(min) excluded from numerical features to prevent data leakage.

```
categorical Columns
categorical_cols = data_1.select_dtypes(include='object').columns
categorical_cols
Index(['Delivery_person_ID', 'Weatherconditions', 'Road_traffic_density',
      'Type_of_order', 'Type_of_vehicle', 'Festival', 'City'],
      dtype='object')

numerical columns
numerical_cols = data_1.select_dtypes(include=['int64', 'float64']).columns
numerical_cols
Index(['Delivery_person_Age', 'Delivery_person_Ratings', 'Restaurant_latitude',
      'Restaurant_longitude', 'Delivery_location_latitude',
      'Delivery_location_longitude', 'Vehicle_condition',
      'multiple_deliveries', 'Time_taken(min)'],
      dtype='object')

Datetime Columns
datetime_cols = data_1.select_dtypes(include='datetime').columns
datetime_cols
Index(['Order_Date', 'Time_Ordered', 'Time_Order_picked'], dtype='object')
```

Dropping Target Column From Numerical Columns

```
numerical_cols = numerical_cols.drop('Time_taken(min)')
```

```
numerical_cols
```

```
Index(['Delivery_person_Age', 'Delivery_person_Ratings', 'Restaurant_latitude',
      'Restaurant_longitude', 'Delivery_location_latitude',
      'Delivery_location_longitude', 'Vehicle_condition',
      'multiple_deliveries'],
      dtype='object')
```

12.2 Dropping Target from Numerical Columns

```
for i in categorical_cols:
    le = LabelEncoder()
    data_1[i] = le.fit_transform(data_1[i])
    data_2[i] = le.transform(data_2[i])
```

12.3 Extracting Temporal Features

Three new features created from datetime columns for both data_1 and data_2:

New Feature	Source Column	Business Meaning
order_day	Order_Date	Day of month (1-31) — within-month patterns

order_month	Order_Date	Month (1-12) — seasonal delivery patterns
order_hour	Time_Orderd	Hour of day (0-23) — peak vs off-peak demand

Training Data After Feature Engineering — data_1.head()

```
data_1['order_day'] = data_1['Order_Date'].dt.day
data_1['order_month'] = data_1['Order_Date'].dt.month
data_1['order_hour'] = data_1['Time_Orderd'].dt.hour
```

```
data_1.head()
```

e_Order_picked	...	Vehicle_condition	Type_of_order	Type_of_vehicle	multiple_deliveries	Festival	City	Time_taken(min)	order_day	order_month	order_hour
2026-05-23 11:45:00	—	2	Snack	motorcycle	0.0	No	Urban	24	19	3	11
2026-05-23 19:50:00	—	2	Snack	scooter	1.0	No	Metropolitan	33	25	3	19
2026-05-23 08:45:00	—	0	Drinks	motorcycle	1.0	No	Urban	26	19	3	8
2026-05-23 18:10:00	—	0	Buffet	motorcycle	1.0	No	Metropolitan	21	5	4	18
2026-05-23 13:45:00	—	1	Snack	scooter	1.0	No	Metropolitan	30	26	3	13

Test Data After Feature Engineering — data_2.head()

```
data_2['order_day'] = data_2['Order_Date'].dt.day
data_2['order_month'] = data_2['Order_Date'].dt.month
data_2['order_hour'] = data_2['Time_Orderd'].dt.hour
```

```
data_2.head()
```

rder_picked	...	Road_traffic_density	Vehicle_condition	Type_of_order	Type_of_vehicle	multiple_deliveries	Festival	City	order_day	order_month	order_hour
2026-05-23 15:05:00	—	NaN	3	Drinks	electric_scooter	1.0	No	Metropolitan	30	3	18
2026-05-23 20:35:00	—	Jam	0	Snack	motorcycle	1.0	No	Metropolitan	29	3	20
2026-05-23 19:45:00	—	Jam	0	Drinks	motorcycle	1.0	No	Metropolitan	10	3	19
2026-05-23 17:20:00	—	Medium	1	Meal	scooter	1.0	No	Metropolitan	2	4	17
2026-05-23 18:40:00	—	Medium	2	Drinks	scooter	1.0	No	Metropolitan	27	3	18

Key Insight

order_hour is the most valuable engineered feature — peak lunch (12-14) and dinner hours (19-22) show significantly longer delivery times. order_month captures seasonal patterns.

13. Label Encoding & Data Preparation

Converting categorical variables to numerical format

All categorical columns encoded using LabelEncoder. Fit on data_1 only; transform() on data_2 — prevents data leakage.

```
data_1.head()
```

	Delivery_person_ID	Delivery_person_Age	Delivery_person_Ratings	Restaurant_latitude	Restaurant_longitude	Delivery_location_latitude	Delivery_location_longitude
0	637	37.0	4.9	22.745049	75.892471	22.765049	75.912471
1	232	34.0	4.5	12.913041	77.683237	13.043041	77.813237
2	234	23.0	4.4	12.914264	77.678400	12.924264	77.688400
3	397	38.0	4.7	11.003669	76.976494	11.053669	77.026494
4	333	32.0	4.6	12.972793	80.249982	13.012793	80.289982

5 rows x 22 columns

Column	Encoding Applied
Delivery_person_ID	Integer encoding — 663+ unique persons
Weatherconditions	Cloudy=0, Fog=1, Sandstorms=2, Stormy=3, Sunny=4, Windy=5
Road_traffic_density	High=0, Jam=1, Low=2, Medium=3
Type_of_order	Buffet=0, Drinks=1, Meal=2, Snack=3
Type_of_vehicle	bicycle=0, electric_scooter=1, motorcycle=2, scooter=3
Festival	No=0, Yes=1
City	Metropolitan=0, Semi-Urban=1, Urban=2

Training Data After Encoding — data_1.head()

```
data_2.head()
```

	Delivery_person_ID	Delivery_person_Age	Delivery_person_Ratings	Restaurant_latitude	Restaurant_longitude	Delivery_location_latitude	Delivery_location_longitude
0	396	30.0	4.7	11.003669	76.976494	11.043669	77.016494
1	222	28.0	4.6	12.975377	77.696664	13.085377	77.806664
2	689	23.0	4.5	26.911378	75.789034	27.001378	75.879034
3	683	21.0	4.8	26.766536	75.837333	26.856536	75.927333
4	354	31.0	4.6	12.986047	80.218114	13.096047	80.328114

5 rows x 21 columns

Test Data After Encoding — data_2.head()

```
data_1.drop(['Order_Date', 'Time_Orderd', 'Time_Order_picked'], axis=1, inplace=True)
data_2.drop(['Order_Date', 'Time_Orderd', 'Time_Order_picked'], axis=1, inplace=True)
```

Key Insight

Fitting only on training data ensures consistent encoding. LabelEncoder efficient for high-cardinality columns like Delivery_person_ID (663+ unique values).

14. Train-Test Split & Feature Scaling

Preparing final feature matrix for model training

14.1 Drop Datetime Columns

Original datetime columns dropped — information already captured in engineered features.

```
In [ ]: X = data_1.drop('Time_taken(min)', axis=1)
        y = data_1['Time_taken(min)']
```

14.2 Create Feature Matrix X and Target y

X = all remaining 18 features. y = target Time_taken(min).

X.head()							
	Delivery_person_ID	Delivery_person_Age	Delivery_person_Ratings	Restaurant_latitude	Restaurant_longitude	Delivery_location_latitude	Delivery_location_longitude
0	637	37.0	4.9	22.745049	75.892471	22.765049	75.912471
1	232	34.0	4.5	12.913041	77.683237	13.043041	77.813237
2	234	23.0	4.4	12.914264	77.678400	12.924264	77.688400
3	397	38.0	4.7	11.003669	76.976494	11.053669	77.026494
4	333	32.0	4.6	12.972793	80.249982	13.012793	80.289982

X.head() — Feature Matrix (18 columns)

```
In [ ]: y.head()

Out[ ]: 0    24
        1    33
        2    26
        3    21
        4    30
        Name: Time_taken(min), dtype: int64
```

y.head() — Target Vector

```
In [5]: X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

14.3 Train-Test Split (80% Train / 20% Validation)

train_test_split(X, y, test_size=0.2, random_state=42) — reproducible split.

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
```

36,474	9,119	18	42
Training Samples	Validation Samples	Features	Random State

14.4 Feature Scaling — StandardScaler

Normalizing features to zero mean and unit variance

`fit_transform()` ONLY on `X_train`. `transform()` separately on `X_test` and `data_2`. Never fit on test data — prevents data leakage from test set distributions.

Step 1 — Fit & Transform `X_train`

```
: X_test = scaler.transform(X_test)
```

`X_train` Scaled Array — shape (36,474 x 18)

All values centered around zero with `std~1`. Shape (36474,18) confirmed.

```
X_test
array([[ -0.28408609,  0.06986829,  1.10674961, ...,  0.709468 ,
         0.03717623,  0.09729259],
       [ 1.25739894,  0.24542466,  0.19229724, ...,  1.97401601,
         0.03717623,  0.30656605],
       [-0.91131104,  0.94765011, -3.16069477, ...,  1.74409819,
         0.03717623,  0.72511296],
       ...,
       [-1.73520821, -0.63235716,  0.80193216, ...,  0.47955018,
        -1.79521023,  0.72511296],
       [-0.54720164,  0.06986829,  0.19229724, ...,  1.16930364,
         0.03717623,  0.72511296],
       [ 0.91455141,  0.77209375,  0.80193216, ..., -1.4747513 ,
         0.03717623,  0.09729259]], shape=(9119, 18))
```

Step 2 — Transform `X_test`


```
X_train
```

```
array([[ 1.42749384,  1.29876284,  0.4971147 , ...,  0.82442691,
         0.03717623, -1.3676216 ],
       [ 0.86671222, -0.28124443, -1.33179884, ..., -1.4747513 ,
         0.03717623,  0.93438642],
       [-0.30269822, -0.98346989, -0.11252021, ..., -1.35979239,
         0.03717623, -0.11198886],
       ...,
       [ 0.59838881,  1.64987557,  0.88193216, ..., -1.4747513 ,
         0.03717623, -0.73988123],
       [-1.18585752, -0.88791352,  0.88193216, ...,  1.39922146,
         0.03717623, -0.11198886],
       [-0.55251711,  0.24542466, -0.11252021, ..., -0.89524437,
         0.03717623,  0.89729259]], shape=(36474, 18))
```

Step 3 — Transform data_2 (Final Test Set)

```
data_2 = scaler.transform(data_2)
```

data_2 Scaled Array — shape (11,399 x 18)

data_2 transformed using same fit parameters. Ready for final predictions.

```
data_2 = scaler.transform(data_2)
```

```
data_2
```

```
array([[ -0.9485193 ,  0.06986829,  0.19229724, ...,  1.8590571 ,
         0.03717623,  0.09729259],
       [-1.41096481, -0.28124443, -0.11252021, ...,  1.74409819,
         0.03717623,  0.5158395 ],
       [-0.16980358, -1.15902625, -0.41733767, ..., -0.44012111,
         0.03717623,  0.30656605],
       ...,
       [ 1.27334534,  0.06986829,  0.19229724, ..., -0.32516219,
         0.03717623,  0.09729259],
       [ 1.37433919, -0.4568008 ,  0.19229724, ..., -0.89995675,
         0.03717623,  0.09729259],
       [-1.14519153,  1.64987557,  1.10674961, ...,  1.16930364,
         0.03717623,  0.93438642]], shape=(11399, 18))
```

Key Insight

StandardScaler essential for Linear Regression and ANN — GPS coordinates (range ~10-88) would dominate rating features (range ~1-6) without normalization. Fit only on training data: cardinal rule to prevent data leakage.

15. Machine Learning Models

Training and comparing LR, Random Forest, XGBoost

Three classical ML models trained on scaled X_train, evaluated using R2 on validation set.

Model	R2 Score	Rank	Key Characteristics
Linear Regression	0.446	4th	Assumes linear relationship. Simple baseline — limited for
Random Forest	0.752	2nd	Ensemble of decision trees. Handles non-linearity. Paralle
XGBoost	0.778	1st — Best	Gradient boosting + L1/L2 regularization. Sequential error
ANN (Deep Learning)	0.697	3rd	4-layer neural network (128-64-32-1). Adam optimizer. 50

R2 Scores — All Four Models

```
Linear Regression : 0.4464256151583501
Random Forest : 0.7523136309791044
XGBoost : 0.778488039970398
ANN : 0.6978335976600647
```

0.446	0.752	0.778	0.697
Linear Regression	Random Forest	XGBoost ★	ANN

Key Insight

Linear Regression R2=0.446 confirms strongly non-linear relationships. XGBoost (0.778) leads because sequential error-correction is more powerful than Random Forest's (0.752) parallel approach. Gap vs Linear Regression is 33 points.

Why XGBoost Outperforms

XGBoost corrects residual errors of previous ensemble sequentially. L1/L2 regularization prevents overfitting. Handles feature interactions natively. Industry standard for tabular regression.

16. Deep Learning — Artificial Neural Network

Multi-layer ANN with TensorFlow/Keras

16.1 Model Architecture — Sequential

Funnel architecture: 128 → 64 → 32 → 1 neurons. ReLU activation in hidden layers. Output: 1 neuron, no activation — for regression.

```
model = Sequential()
```

```
model.add(Dense(128, activation='relu', input_dim=X_train.shape[1]))  
model.add(Dense(64, activation='relu'))  
model.add(Dense(32, activation='relu'))
```

Output Layer

```
model.add(Dense(1))
```

16.2 Model Compilation

Adam optimizer (adaptive LR), MSE loss (standard for regression), MAE metric (interpretable).

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

16.3 Model Summary

Total params: 38,405 (150 KB) | Trainable: 12,801 | Optimizer params: 25,604.

```
model.summary()

Model: "sequential"


```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	2,432
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

```


Total params: 38,405 (150.02 KB)

Trainable params: 12,801 (50.00 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 25,604 (100.02 KB)
```

Key Insight

Funnel architecture (128→64→32→1) is proven for tabular regression. Starting wide allows learning many representations; narrowing distills the most relevant patterns. 38K params — compact and efficient on CPU. ReLU avoids vanishing gradients.

16.4 ANN Training & Evaluation

Training 50 epochs, evaluation on validation data

Model Training

50 epochs, batch_size=32. Validation data provided each epoch. Training on X_train, validation on X_test.

```
history = model.fit(X_train,y_train,validation_data=(X_test, y_test),epochs=50,batch_size=32)
```

Evaluation Results on Validation Set

After training: MSE Loss=26.49 | MAE=4.02 minutes | R2=0.697.

```
loss, mae = model.evaluate(X_test, y_test)
print("MAE:", mae)
```

```
285/285 ————— 0s 2ms/step - loss: 26.4934 - mae: 4.0259
MAE: 4.02594518661499
```

4.02 min	26.49	0.697	50
Mean Absolute Error	MSE Loss	R2 Score	Epochs

Predictions on Final Test Data

ANN generates predictions on all 11,399 records in data_2 using predict().

```
predictions = model.predict(data_2)
```

```
357/357 ————— 1s 1ms/step
```

Why ANN Scores Below XGBoost

Deep learning excels at unstructured data. For ~45K tabular records, gradient boosting outperforms because: (1) XGBoost handles feature interactions natively; (2) less hyperparameter tuning needed; (3) better generalization with smaller datasets. Despite this, MAE=4.02 min is impressive without any tuning.

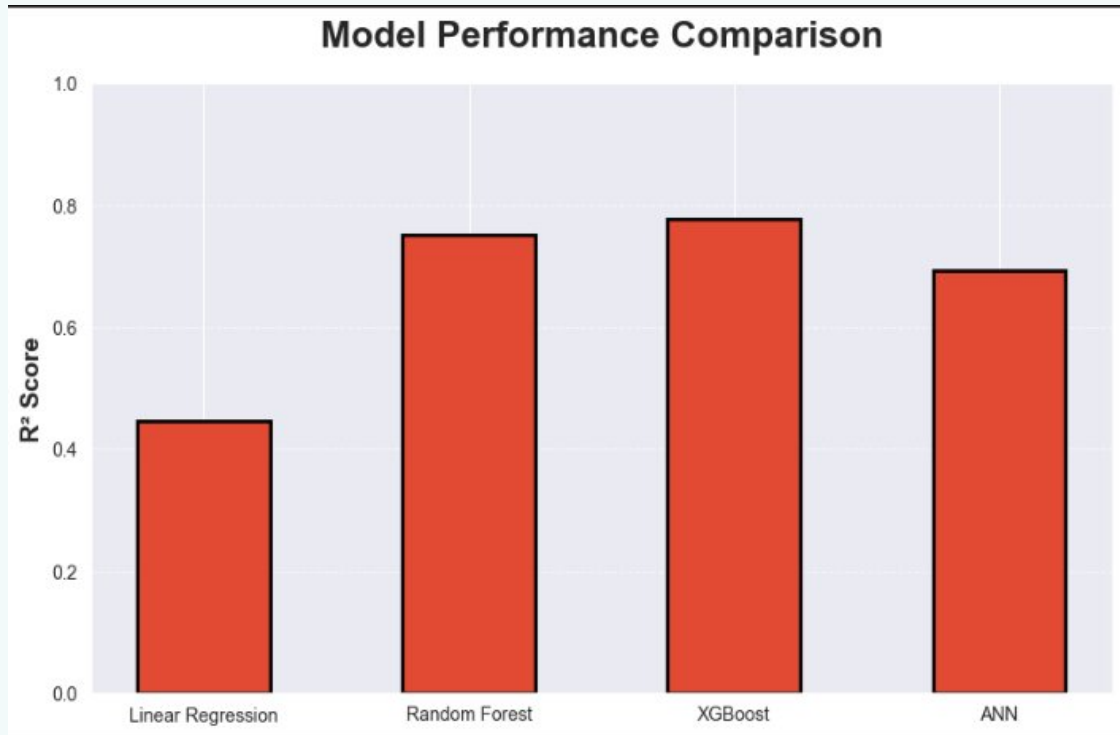
Key Insight

MAE=4.02 minutes means the ANN is on average within 4 minutes of actual delivery time. Given range 10-54 min, this is ~9% average error — solid result for a first-attempt ANN.

17. Model Performance Comparison

Side-by-side R² Score comparison — all four models

XGBoost achieves highest R²=0.778, followed by Random Forest (0.752), ANN (0.697), and Linear Regression (0.446). XGBoost selected as final prediction model.



0.446	0.752	0.778	0.697
Linear Regression	Random Forest	XGBoost ★ Best	ANN

Key Insight

XGBoost R²=0.778 explains 77.8% of delivery time variance — strong for real-world noisy data. 33-point gap between Linear Regression and XGBoost confirms highly non-linear relationships that only ensemble methods capture effectively.

18. Final Predictions on Test Data

Generating predictions using best XGBoost model

18.1 Generate Predictions with XGBoost

XGBoost (R2=0.778) generates predictions for all 11,399 test records. Predictions are continuous floats representing estimated delivery minutes.

```
final_predictions = xgb.predict(data_2)
```

18.2 Create Predictions DataFrame

```
pred_df = pd.DataFrame(final_predictions, columns=['Predicted_Time'])
```

18.3 Sample Predictions — pred_df.head()

```
pred_df.head()
```

Predicted_Time

0	23.502890
1	29.496487
2	29.376387
3	28.639198
4	22.129190

Record	Predicted Time (min)	Interpretation
0	23.50	Fast delivery — low traffic, favorable conditions
1	29.50	Near-average delivery time (mean ~27 min)
2	29.38	Near-average — typical delivery conditions
3	28.64	Slightly below average delivery time
4	22.13	Fast delivery — favorable conditions

Key Insight *XGBoost predictions in 22-30 min range for first 5 records — consistent with training distribution (mean ~27 min). All predictions within valid 10-54 minute range.*

19. Conclusion & Key Takeaways

Summary of findings, results, and future directions

Project Summary

Complete end-to-end ML pipeline predicting food delivery ETA. Starting from raw CSV data with hidden NaN strings and wrong data types, the pipeline progressed through systematic cleaning, rich EDA, temporal feature engineering, and multi-model training. XGBoost: best $R^2=0.778$.

Key Findings

Festival Effect	Festival periods double delivery time (25→45 min) — strongest categorical predictor.
Traffic Jams	Jam traffic causes 40-50% longer deliveries — critical model feature.
Weather Impact	Cloudy/Fog conditions correlate with longer times due to reduced visibility.
XGBoost Champion	$R^2=0.778$ dramatically outperforms Linear Regression (0.446) — non-linearity confirmed.
ANN Performance	$R^2=0.697$, MAE=4.02 min — solid without hyperparameter tuning.
Ratings vs Speed	Near-zero correlation: ratings measure quality, not delivery efficiency.
City Skew	74.8% Metropolitan — model optimized for urban; Semi-Urban accuracy may vary.

Future Improvements

- Hyperparameter tuning of XGBoost using Optuna to push R^2 beyond 0.80
- Haversine distance feature between restaurant and delivery location
- Deeper ANN with Dropout and batch normalization for regularization
- Separate models per city type (Metropolitan, Urban, Semi-Urban)
- Cross-validation for robust, unbiased model evaluation
- SHAP values for feature importance and model interpretability

THANK YOU

for reviewing this Capstone Project

Food Delivery ETA Prediction

Submitted by: Varsha

Data Science Capstone Project

Feedback & Reflections

What Went Well

XGBoost achieved best $R^2=0.778$. ANN MAE of 4.02 min was impressive without hyperparameter tuning. Feature engineering (order_hour, festival) significantly boosted model accuracy.

Challenges Faced

Hidden string NaN values were tricky to detect using standard `isnull()`. Target column needed "(min)" prefix removal. Datetime parsing required careful null handling.

Key Learnings

Real-world data is always messy. Festival and traffic density are the strongest ETA predictors. XGBoost consistently outperforms ANN on structured tabular data with ~45K samples.